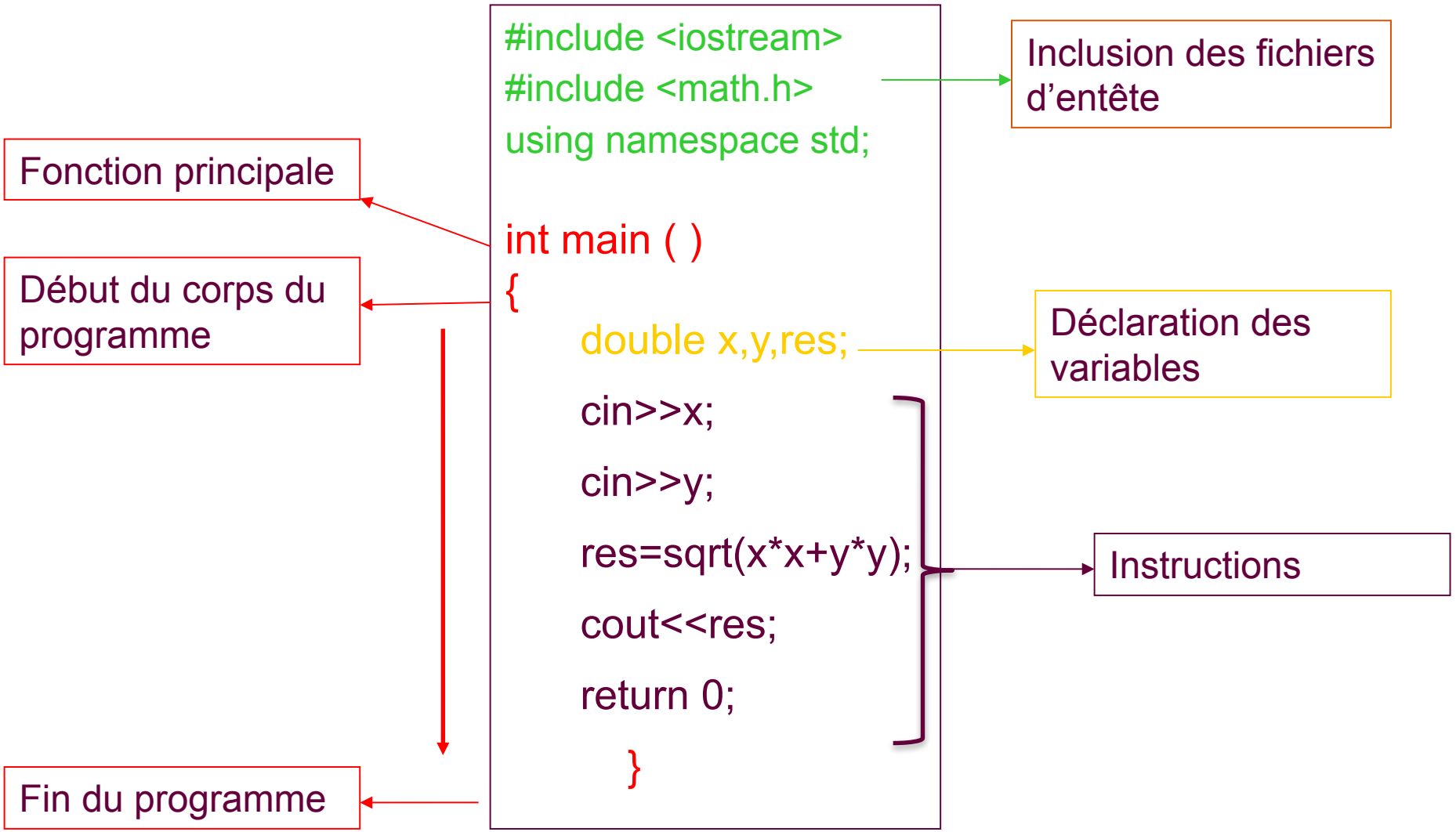
université PARIS-SACLAY | IUT DE CACHAN

Programmation C / C++:
#01 – Rappels de base

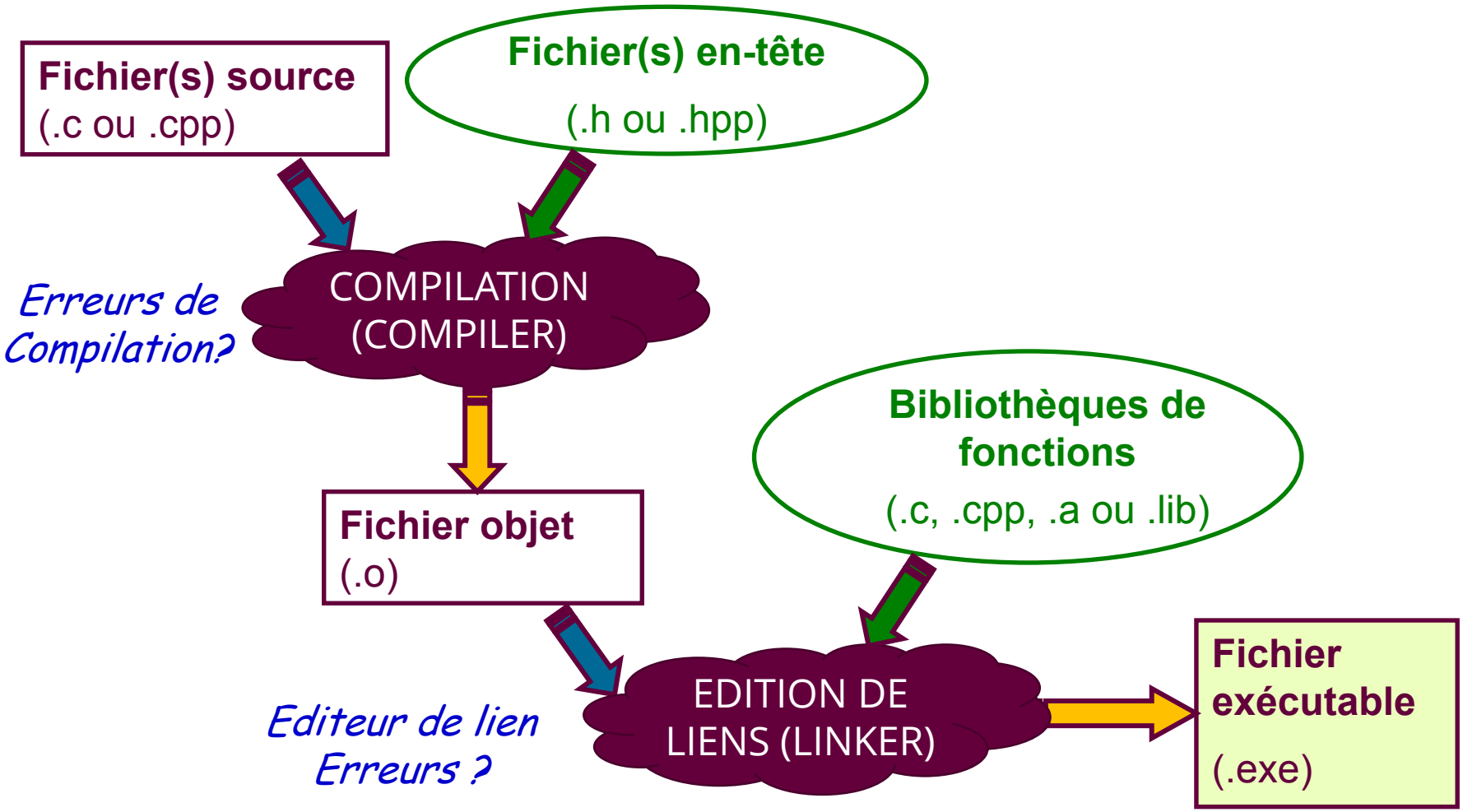


BUT 3 ESE – LP MECSE
v1.8 – Sept. 2025

Rappel

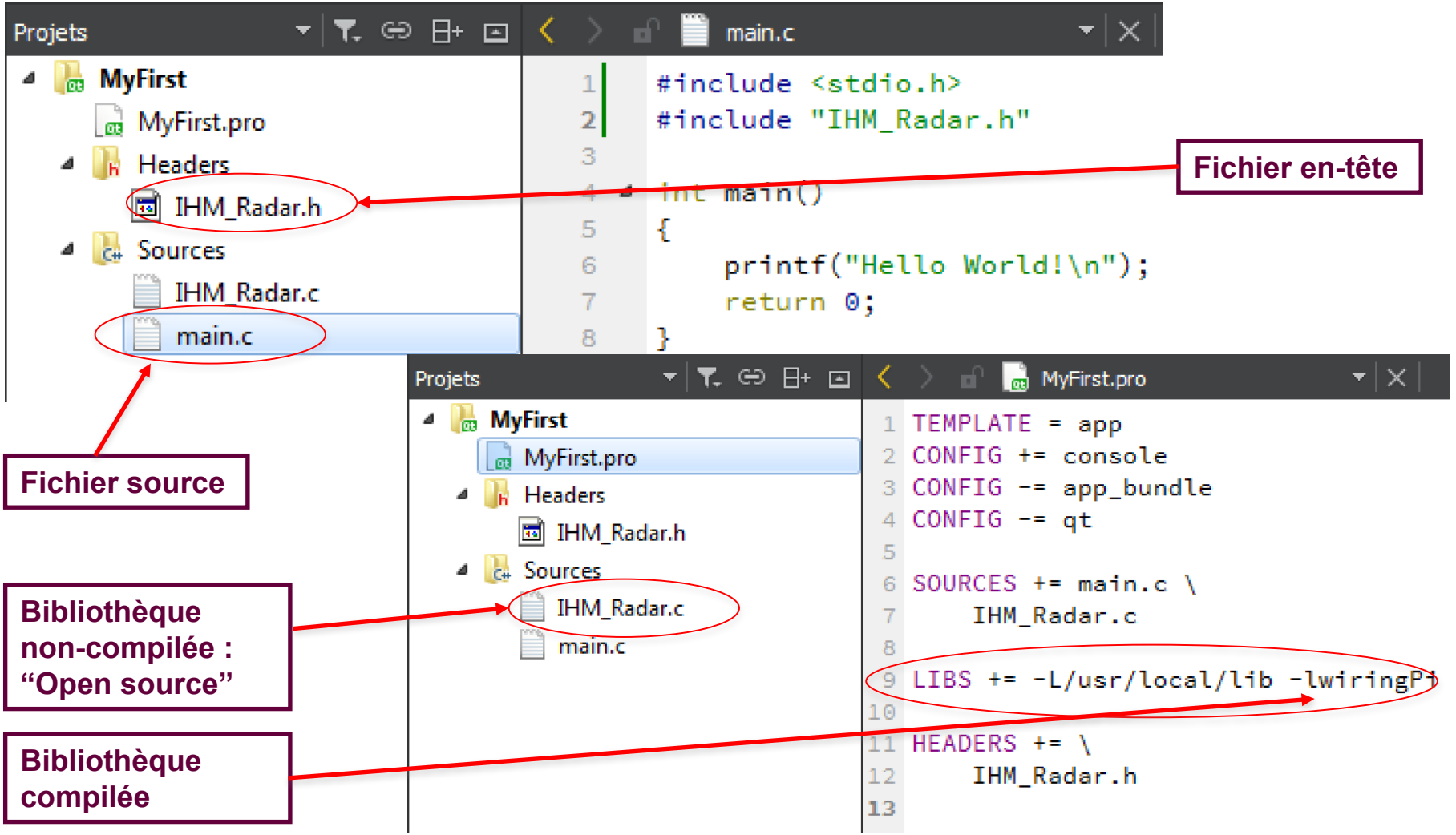

#include <iostream>
#include <math.h>
using namespace std;
int main ( )
{
double x,y,res;
cin>>x;
cin>>y;
res=sqrt(x*x+y*y);
cout<<res;
return 0;
}
Inclusion des fichiers d'entête
Fonction principale
Début du corps du programme
Déclaration des variables
Instructions
Fin du programme

Génération d'un exécutable



⇒ Travail sous forme de **projets**

Exemple : Projet Qt



Comparatif C/C++ saisie/affichage

```c
#include <stdio.h>


int main()
{
   int age;

   printf("Votre age ?\n");
   scanf("%d", &age);
   printf("Vous avez %d ans !\n",age);

   return 0;
}
```

```cpp
#include <iostream.h>
using namespace std;

int main()
{
   int age;

   cout << "Votre age ?"<<endl;
   cin >> age;
   cout << "Vous avez "<< age <<"ans" << endl;

   return 0;
}
```

Exemple complet (en C++)

```
#include <iostream>

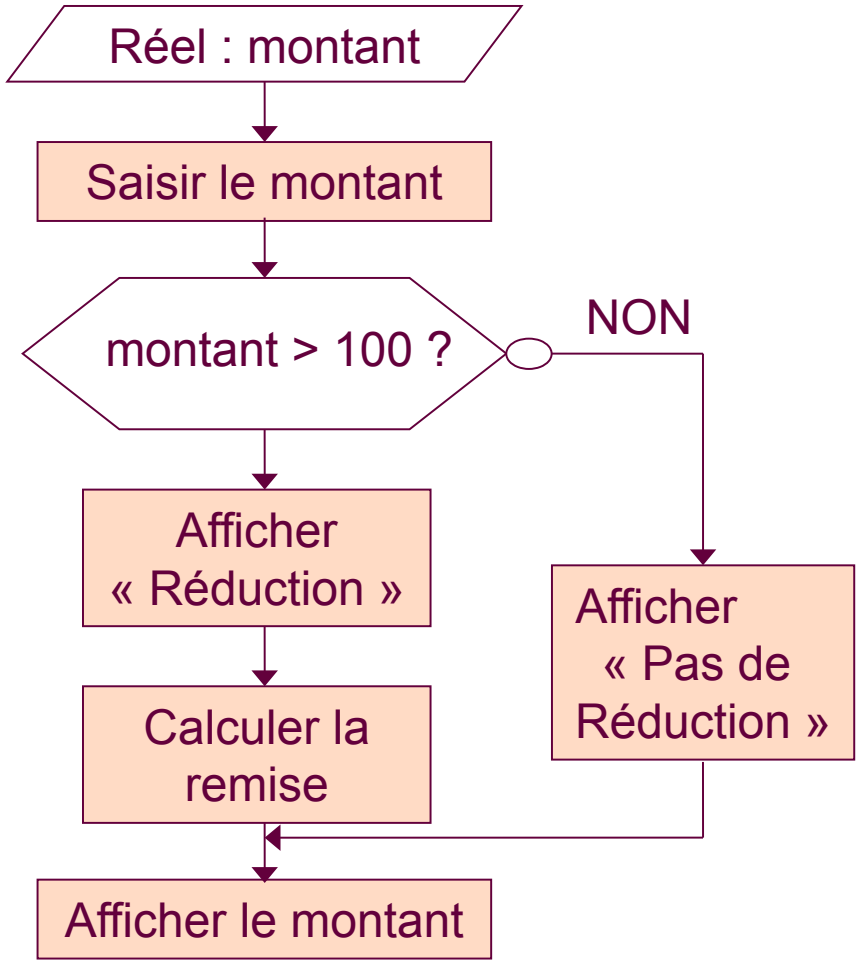
using namespace std;

int main(void)
{
    int valMax=34 , nouvelleVal;
    cout << "Entrez un nombre:";
    cin >> nouvelleVal;
    if (nouvelleVal >valMax) //Nouveau Max?
    {
   // Mise à jour de valMax avec la saisie
       valMax = nouvelleVal;
       cout << "Nouveau Maximum :" << valMax;
    }
    return 0;
}
```

L'Alternative "si…alors…sinon"

```
DEBUT
  REEL montant
  SAISIR(montant)
  SI (montant>100)
  ALORS
     AFFICHER(« reduction »)
     montant←montant-0.1*montant
  SINON
     AFFICHER(« pas de reduction »)
  FINSI
AFFICHER(montant)
FIN
```

Organigramme correspondant



Petit casse-tête !

```
double x=-5,y=2;
if (x==0)
if(y>0)
cout<<"cas"<<endl;
if(x<0)
cout<<"cas 2"<<endl;
else
cout<<"cas 3"<<endl;
cout<<"cas 4"<<endl;
```



Quels sont les affichages obtenus?

```
cas 2
cas 4
```

Et en mieux…

```
double x=-5,y=2;
if (x==0)
  if(y>0)
    cout<<"cas 1"<<endl;
if(x<0)
  cout<<"cas 2"<<endl;
else
  cout<<"cas 3"<<endl;
cout<<"cas 4"<<endl;
```

Il faut **indenter** (et commenter) les programmes

Il faut des **accolades** si il y a plusieurs instructions à réaliser

L'instruction « **else** » se rapporte au « **if** » le plus proche

Cas particulier : “switch()…case”

Exécuter des instructions **en fonction de** la valeur (***entière***) d’une condition

```
switch (variable)
{
   case constante1:
        instructions1;
        break;
   case constante2:
        instructions2;
        break;
  […] //nb de ‘case’ illimité
   default:
        instructions_par_defaut;
}
```

Exemple

```
char operateur;
int x=5,y=2,res;
cin>>operateur;
switch (operateur)
{
   case '+':   res=x+y;
        break;
   case '-':   res=x-y;
        break;
   case '*':   res=x*y;
        break;
   case '/ ':  res=x/y;
        break;
   default:
        cout<<"opérateur inconnu";
}
cout<<res;
```

Instructions répétitives

Dans de nombreux cas, il est nécessaire d'exécuter plusieurs fois une instruction ou un bloc d'instructions au sein d'un programme
➔ Mise en place de boucles permettent de faire ceci sans réécrire les instructions correspondantes

Le langage C dispose des structures répétitives (boucles) suivantes :

- *Boucle "while"*

- *Boucle "do while"*

- *Boucle "for"*

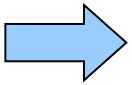
Boucle “do while”

But : répéter une action ... au moins une fois !!

Faire

instructions

Tant Que (condition VRAIE)



```
do
{
   instructions;
} while (condition);
```

!!

- *Le corps de la boucle "do while" est répété tant que la condition est évaluée comme vraie (différente de 0)*

- On exécute le corps de la boucle PUIS on teste la condition pour un éventuel rebouclage

- Le corps de la boucle est toujours exécuté au moins une fois même si la condition est fausse !!

Exemple 1

Tester les entrées …

```
double resistance;
do
{
  cout<<''valeur de la resistance?";
  cin>>resistance;
} while (resistance<=0);
// ici la resistance>0
cout<<''R = ''<<resistance;
```

Permet d'être sûr d'avoir une valeur de résistance positive pour effectuer le reste du programme

Exemple 2

Recommencer un programme

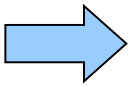
```
char choix;
do
{
  […] // instructions du programme
  cout<<''voulez-vous recommencer?'';
  cin>>choix;
} while (choix=='o');
```

Le programme est exécuté une première fois, et on recommence tant que l'utilisateur le souhaite.

La boucle “while”

!!

TantQue (condition
VRAIE)
Faire
instructions

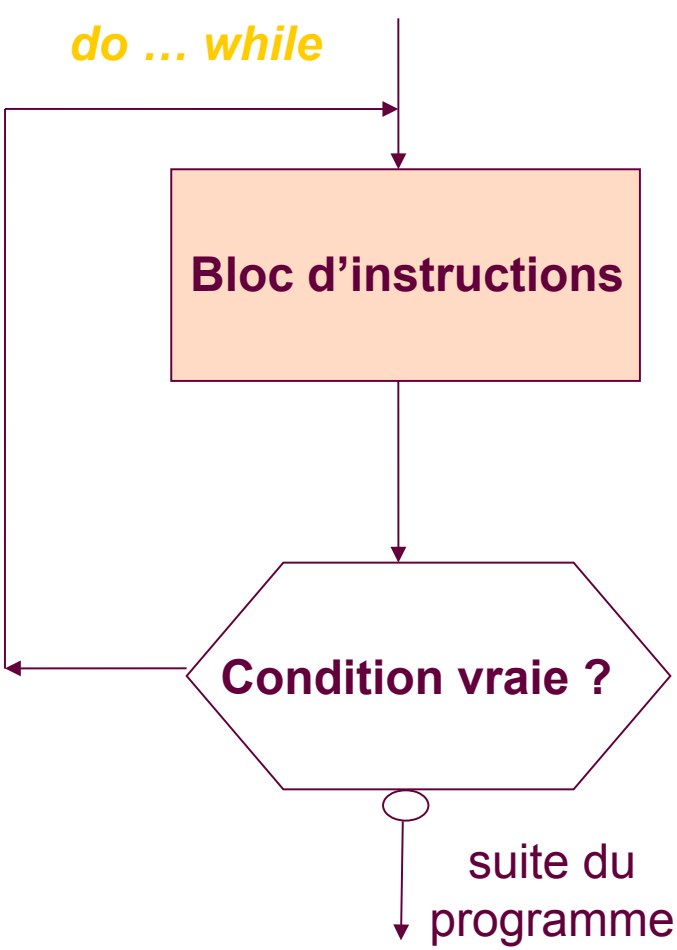


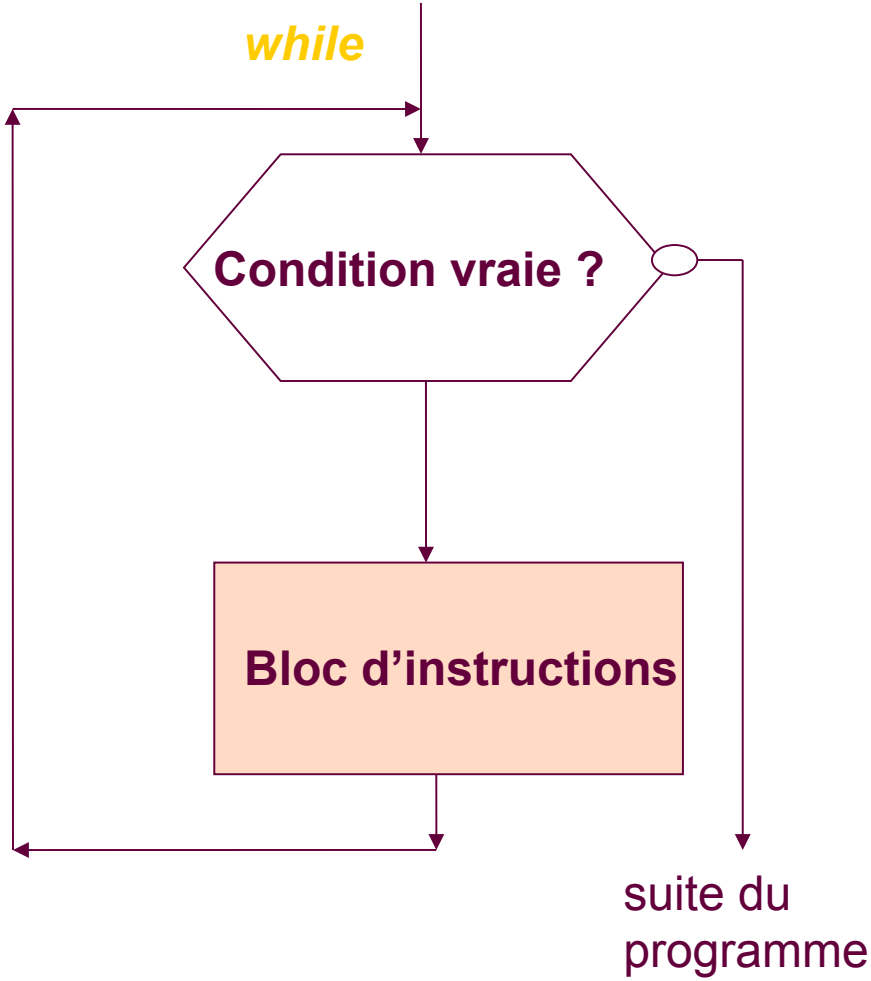
```
while(condition)
{
    instructions;
}
```

- *Le corps de la boucle "while" est répété tant que la condition est évaluée comme vraie (différente de 0)*
- *On teste D’ABORD la condition pour une éventuelle exécution du corps de la boucle et on revient au test*

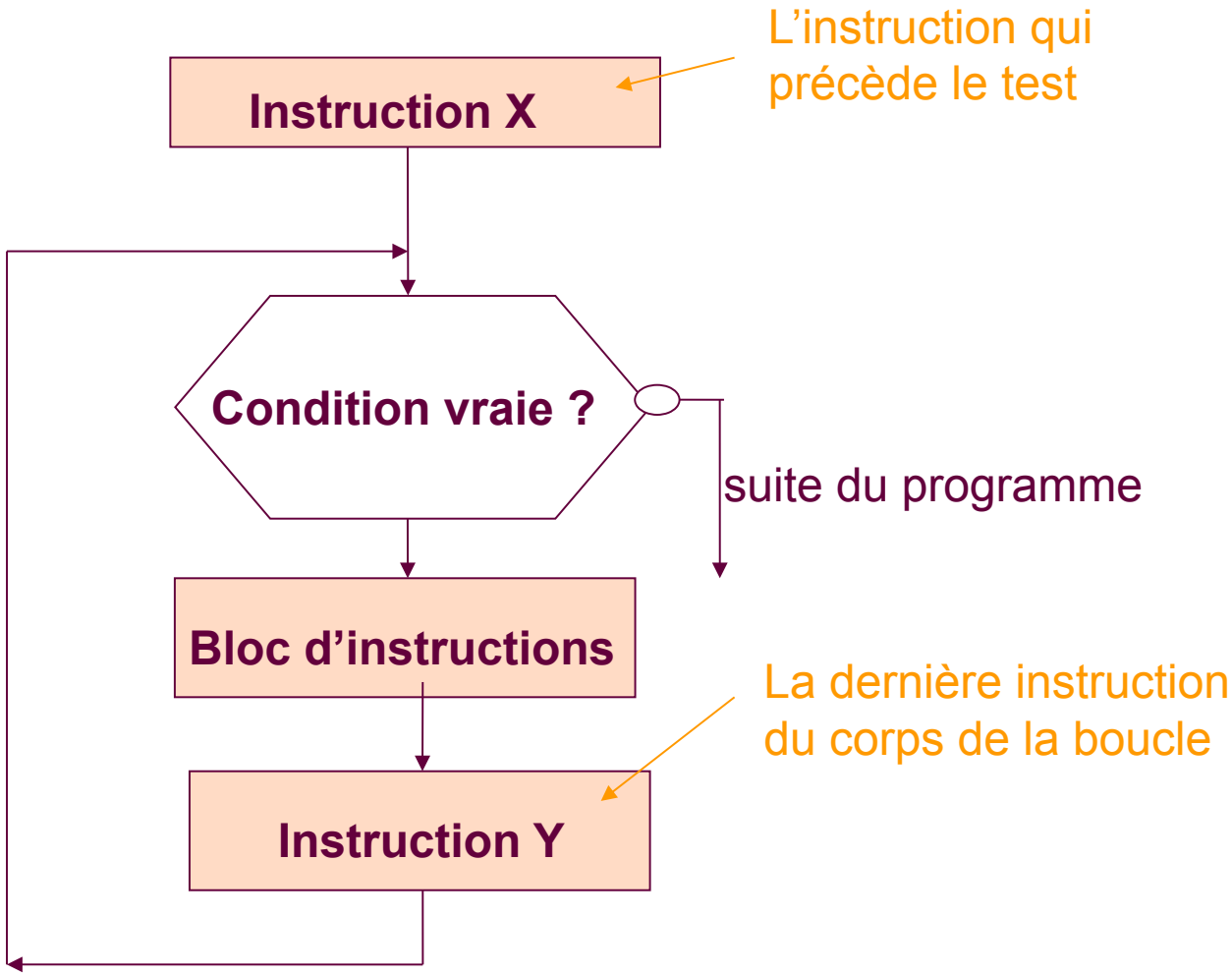
*- Si le test est faux dès le départ, le corps de la boucle n’est **jamais** exécuté*

“while” vs “do while”





Boucle “for”


L’instruction qui précède le test
Instruction X
Condition vraie ?
suite du programme
Bloc d’instructions
La dernière instruction du corps de la boucle
Instruction Y

Intérêt de la boucle “for”

Afficher les entiers de 1 à 100

Version 1 « while »

```
int i;

i=1;
while(i<=100)
{
   cout<<i<<endl;
   i++;
}
suite du programme…
```

Version 2 « for »

```
int i;

for(i=1;i<=100;i++)
{
   cout<<i<<endl;
}
suite du programme…
```

i est appelé « index » ou « compteur de boucle »

Problème…

(t_0, t_1, … , t_{n-1}) → n valeurs à traiter
(ex: acquisitions multiples d'un capteur)

2 solutions :

- Déclarer n variables et les traiter indépendamment… LOURD!
- Utiliser un tableau :
  - Une seule déclaration
  - Un traitement simplifié via des boucles!

Propriétés – Déclaration

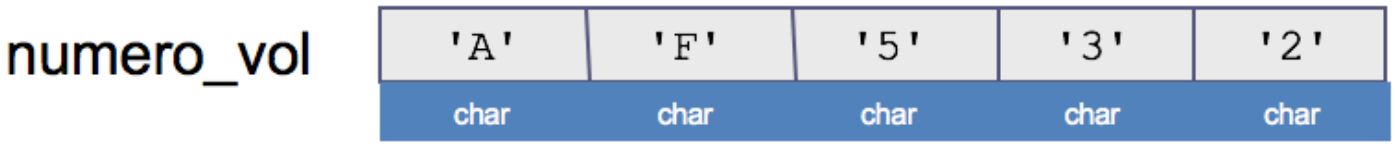
- Stocker et manipuler des éléments de même type
- Nombre d'éléments fini (nombre d'éléments du tableau)
- Chaque élément est manipulé à l'aide de son indice

```
type_elements  nom[nbr_elements];
```

Constante!

Exemples:

```
char numero_vol[5]={'A','F','5','3','2'};
```



Un tableau de 25 notes : `float notes[25];`

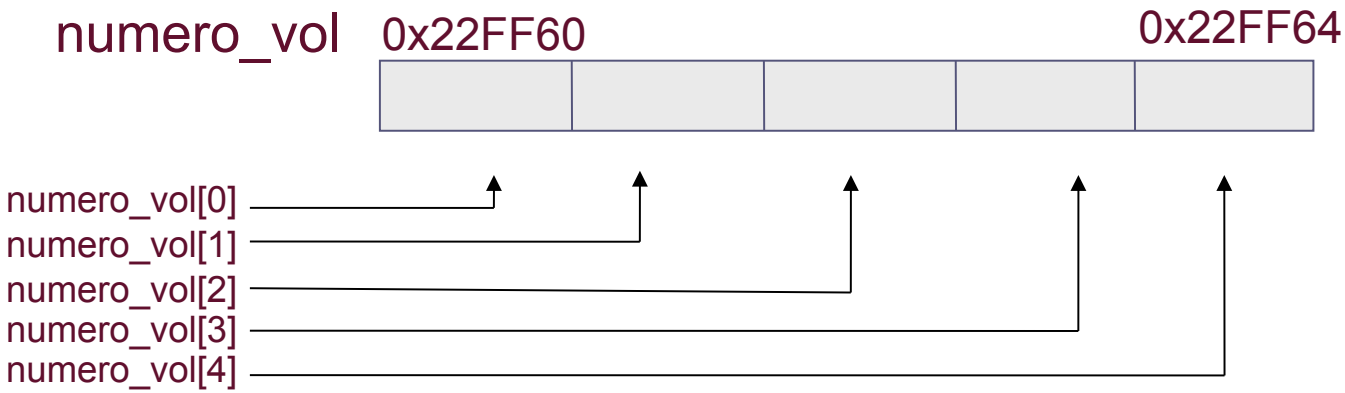
Allocation mémoire *Statique*

Un tableau occupe un bloc mémoire de taille égale au nombre d'éléments x taille d'un élément

L'adresse de ce bloc correspond à l'adresse du 1er élément

Exemples :

float notes[25];	25x4=100 octets
char numero_vol[5];	5x1=5 octets



Initialisation d'un tableau

Comme pour les variables, par défaut, les éléments d'un tableau ont des valeurs indéterminées

On peut préciser les valeurs des éléments à la définition

Exemples :

```
int tab[5]={0,1,-1,2,1};
```

```
float temperatures[30]={12,8,7,8.5,11,14,15};
```

```
char numero_vol[5]={'A','F','5','3','2'};
```

Note : Les valeurs non précisées sont alors mises à zéro

Accès aux éléments

→ se fait grâce à l'indice de l'élément :
un tableau de n éléments est indicé de 0 à n-1. 

nom_tableau[i] correspond à l'élément i+1 du tableau

Exemples :

```
char numero_vol[5] = {'A' , 'F' , '5' , '3' , '2' };
```

numero_vol[0] a pour valeur 'A'

numero_vol[4] a pour valeur '2'

numero_vol[5] a pour valeur 0

*Remarque : numero_vol contient l'**adresse** du premier élément, et l'indice représente le déplacement par rapport à cet élément*

Parcours d'un tableau

Exemple : c'est l'utilisateur qui saisit les valeurs du tableau

Version 1

```
int main()
{
     int tab[5];
     scanf("%d",&tab[0]);
     scanf("%d",&tab[1]);
     scanf("%d",&tab[2]);
     scanf("%d",&tab[3]);
     scanf("%d",&tab[4]);
    }
```


Version 2

```
int main()
{
     int tab[5];
     int i;
     for(i=0;i<5;i++)
     {
        scanf("%d",&tab[i]);
     }
    }
```

Similaire pour l'affichage d'un tableau en utilisant la fonction printf

Complément
Tableaux à deux dimensions

Définition :

```
type_elements nom[nb_lignes][nb_colonnes];
```

Exemples :

```
int matrice[3][3];
float releve[12][31];

unsigned char imageNetB[480][640];
```



Exemple de tableau à 3 dimensions

La couleur d'un pixel de coordonnées (i,j) est déterminée à l'aide de 3 nombres qui donnent la quantité de rouge, vert et bleu (codage RGB)



On pourrait stocker cette image à l'aide du tableau suivant :

```
unsigned char image [480][640][3];
```

Manipulation des tableaux 2D

Soit T un tableau à 2 dimensions :

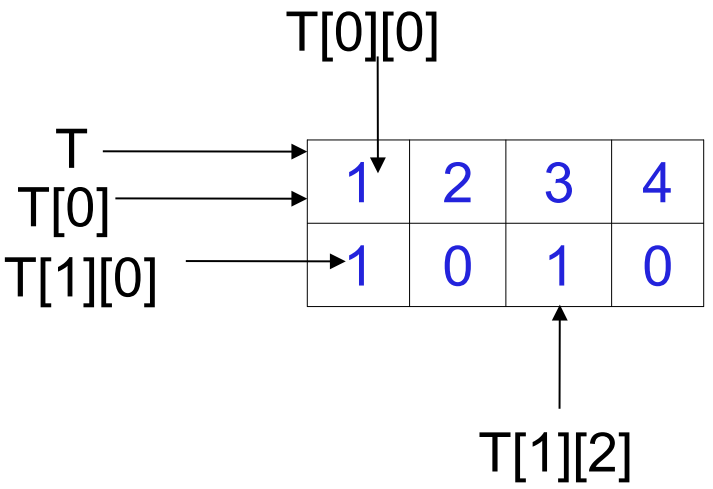
T est l'adresse du premier élément T[0][0],

T[0] est le tableau correspondant à la première ligne,

T[i][j] représente l'élément indicé (i,j)

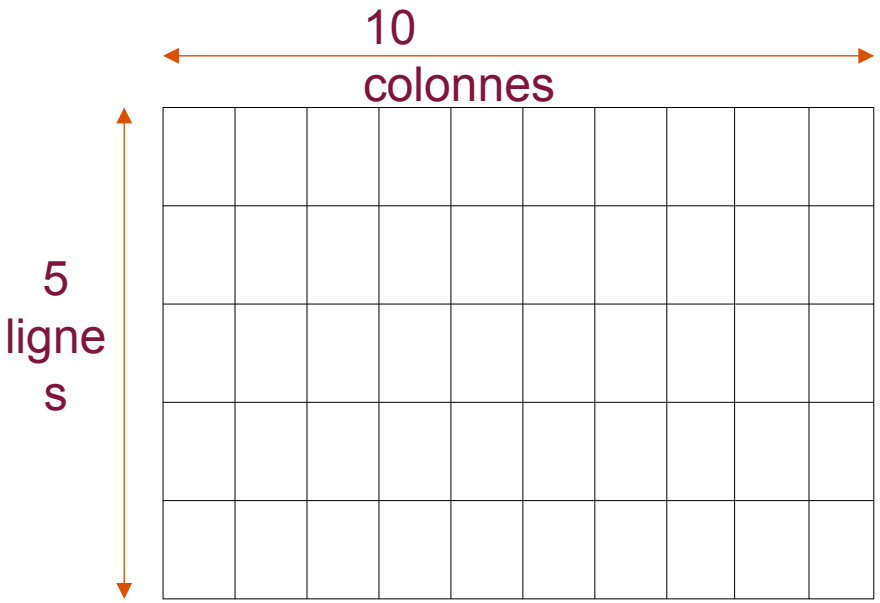
Exemple :

```
int T[2][4]={{1,2,3,4},
             {1,0,1,0}};
```



Exemple de manipulation : Saisie

On veut remplir un tableau 2D d'entiers via des saisies au clavier




```
int main()
{
 int i,j;
 int T[  ][  ];
 for (i=0;i<  ;i++){
    for(j=0;j<  ;j++){
        scanf("%d",&T[ ][ ]);
    }
 }
}
```

Comment devrait-on nommer i et j?

Définition de chaînes de caractères en C

Une chaîne de caractère est un tableau de … caractères !

Exemple : char formation[15] = "Sesam" ;

En mémoire :

Hexa														
ASCII														



'\0' symbolise la fin de la chaîne

prévoir une case de plus pour ce caractère

Parcours spécifique des chaînes




```
int i;
for (i=0; chaine[i]!=        ; i++)
{
    […]  // traitement du caractère chaine[i]
}
```

Exemple : taille de la chaine

```
...

int nb=0;

while (prenom[nb]!='\0')  // parcours de la chaine
{
    nb++;
}

printf("%d caracteres\n", nb);

...
```

Pourquoi?

Spécificités des chaînes

```
char mot[50];
scanf("%s",mot);    // saisie d'une suite de
                    // caractères (49 au maximum !)
```

Contrairement aux tableaux de nombres, on peut donc saisir **directement** une chaîne de caractères

La saisie s'arrête dès qu'on tape sur entrée ou qu'on rencontre un **espace**…

Remarque : pour saisir une phrase par exemple, on utilisera la fonction *fgets* ex : *fgets(**mot**,sizeof **mot**,*stdin*);*

Note : l'opérateur **`sizeof`** *permet d'obtenir la taille d'une zone mémoire*
Ex: **`sizeof mot`**

sprintf : variables & chaines de caractères

Possibilité de fabriquer une *chaine* avec des **caractères** et des **valeurs numériques** issues de variables

Exemple :

```
char tab [50];
  int n, a=5, b=3;
  n=sprintf (tab, "%d plus %d vaut %d", a, b, a+b);
  printf (" Calcul : %s", tab);
```

La variable *n* contiendra alors la longueur de la chaine obtenue

Autre parcours possible

En utilisant la fonction *strlen* de la bibliothèque *string.h*

Exemple :

```
char mot[10]="à vous...";
int i;

for (i=0; i<strlen(mot); i++)
{
    // traitement ...
}
```

*Note1 : en C++, on peut utiliser un "type" nommée **string** qui permet de manipuler les chaines de caractères de manière intuitive*

*Note2 : Qt propose un autre "type", nommée **QString**, plus adapté au framework Qt*

Recommandations

Pour devenir un bon programmeur…

- De la **méthode**…
- Des **commentaires** !
- **Clarté** (choix des noms…)
- **Indentation**



+

